MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Information Technology

TRAFFIC SIGN RECOGNITION USING DEEP LEARNING

MINOR PROJECT SYNOPSIS

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Student 1	Akshit Mehta	Enrollment No.: 00314803123
Student 2	Shashwat Kumar	Enrollment No.: 00414803123
Guide	Mr. Ajay Kaushik	Assistant Professor, Department of Information Technology

Submitted in partial fulfillment of the requirements for the Semester VII Minor Project

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY (Affiliated to Guru Gobind Singh Indraprastha University, Delhi)

Traffic Sign Recognition Using Deep Learning

PROJECT SYNOPSIS

1. Project Title

Traffic Sign Recognition Using Deep Learning

2. Academic Context

This synopsis is prepared for the Semester VII Minor Project of the B.Tech. Information Technology programme at Maharaja Agrasen Institute of Technology. MAIT's Semester VII minor-project guidance requires departmental approval and a synopsis in the prescribed format, while the GGSIPU syllabus available through MAIT's syllabus page requires a synopsis at the beginning of the semester followed by seminars and progress reports. The project is supervised by Mr. Ajay Kaushik, identified in MAIT's official Information Technology records as an Assistant Professor. This synopsis records the two students provided by the team and does not include a project ID because none has been assigned.

3. Problem Statement

Traffic signs communicate rules and warnings, but recognition is difficult when signs are small, tilted, blurred, unevenly illuminated, or visually similar to other classes. The German Traffic Sign Recognition Benchmark contains 43 categories and real-world variation, making it a useful computer-vision classification problem. The project addresses the classification task: given one cropped traffic-sign image, predict its class and confidence. Full-scene object detection and safety-critical driving decisions are outside the scope of this version.

4. Project Abstract

This project proposes a reproducible TensorFlow/Keras pipeline for the 43-class German Traffic Sign Recognition Benchmark. The complete archive will be downloaded from Zenodo, verified with the published MD5 checksum, and checked for every class ID from 0 through 42. The proposed pipeline will perform exploratory data analysis, grouped validation splitting, resizing, normalization, moderate training-only augmentation, model training, official-test evaluation, error analysis, Grad-CAM visualization, and single-image inference. A compact custom CNN will be developed as the baseline, and MobileNetV2 transfer learning will be investigated as a comparison model. The expected outcome is an auditable traffic-sign classification system with documented limitations and reproducible experiment instructions.

5. Literature Survey

The project design is based on verified primary papers and official documentation. Stallkamp et al. define GTSRB as a multi-category traffic-sign benchmark with changes in distance, illumination, weather, occlusion, and rotation; this motivates the dataset, EDA, and augmentation strategy. Sandler et al. describe MobileNetV2's inverted residuals, linear bottlenecks, and depthwise convolutions; this motivates a lightweight transfer-learning comparison. Kingma and Ba introduce Adam, used as the practical baseline optimizer. Selvaraju et al. introduce Grad-CAM, used to visualize regions contributing to a prediction. Official scikit-learn, TensorFlow, and Keras documentation support grouped validation, streaming input pipelines, augmentation, early stopping, learning-rate reduction, and checkpoint saving.

Source	Verified contribution	Project use
Stallkamp et al. 2011	GTSRB benchmark and visual variation	Dataset boundary and evaluation
Sandler et al. 2018	Efficient MobileNetV2 architecture	Transfer-learning comparison
Kingma and Ba 2015	Adaptive Adam optimizer	Training optimizer
Selvaraju et al. 2017	Gradient-based visual explanations	Grad-CAM diagnostic
Official documentation	Grouped split, tf.data, augmentation, callbacks	Validation and pipeline design

6. System Specifications

The proposed training environment will use Google Colab with a visible GPU, while local development and inference may be carried out on an Apple M1 environment. The dataset and generated model artifacts will remain outside Git so that the repository stays lightweight and reproducible.

Parameter	Specification	Operational purpose
Planned benchmark hardware	Google Colab GPU	Accelerated training and evaluation
Local hardware	Apple M1, 8 GB unified memory	Development, smoke tests, inference
Dataset	39,209 train; 12,630 official test	43-class recognition benchmark
Storage policy	Dataset and artifacts ignored by Git	Keep repository reproducible and lightweight

7. Software and Environment Specifications

Software / library	Version / platform	Role
Python	3.11 project environment	Reproducible execution
TensorFlow / Keras	GPU-capable Colab runtime	Training and inference
NumPy / pandas	Pinned project dependencies	Arrays and label tables
scikit-learn	GroupShuffleSplit and metrics	Validation and evaluation
Matplotlib / Pillow / OpenCV	Pinned project dependencies	EDA, images, plots, Grad-CAM

8. Methodology

The archive will be downloaded and verified before extraction. Training labels will be inferred from class directories, while official test labels will be read from Test.csv. Related sequence-like identifiers will be grouped before creating a validation split to reduce near-duplicate leakage. Images will be resized to 64 × 64 for the custom CNN and 128 × 128 for MobileNetV2, converted to RGB, and normalized to [0, 1]. Rotation, translation, zoom, contrast, and brightness transformations will be applied only during training. The planned training process will use early stopping, learning-rate reduction, and best-checkpoint saving.

Stage	Implementation	Reason
Acquisition	Zenodo download plus MD5 verification	Trust the input archive
EDA	Class counts, sample grid, dimensions, brightness, color	Understand data variation
Validation	Grouped 85/15 split from training data	Reduce sequence leakage
Baseline	Compact CNN from scratch	Transparent reference model
Comparison	ImageNet MobileNetV2 with frozen backbone	Lightweight transfer-learning baseline
Training control	Adam, early stopping, LR reduction, checkpoint	Stable and recoverable training
Interpretability	Grad-CAM over a final convolutional layer	Inspect image regions used

9. Expected Outputs

The proposed experiments will generate a Keras model, training history, learning curves, classification report, confusion matrix, and error-analysis plot for each selected model. The analysis will compare accuracy, macro F1, per-class performance, and confusion patterns. The planned Colab notebook will download the dataset directly, train both models with GPU support, and package the generated models and reports into a downloadable ZIP.

Model	Input / run	Measured official-test result
Custom CNN	64 × 64; configurable Colab GPU training	Accuracy, macro F1, per-class metrics, and confusion matrix
MobileNetV2	128 × 128; configurable Colab GPU training	Accuracy, macro F1, per-class metrics, and confusion matrix

10. Scope and Limitations

The proposed work will focus on classification of cropped sign images and will not claim to be a complete autonomous-driving component. Full-scene object detection, camera tracking, and safety-critical driving decisions will remain outside the scope. The comparison between a custom CNN and MobileNetV2 will be interpreted only within the selected training settings. Possible future extensions include calibration, out-of-distribution detection, fine-tuning, real-time detection, and real-world safety validation.

11. Project Deliverables

The project is expected to produce a verified dataset downloader, reusable TensorFlow modules, command-line training/evaluation/inference interfaces, EDA and reporting scripts, tests, a standalone Colab notebook, an interactive dashboard, Grad-CAM visualizations, an IEEE-style report, a presenter guide, and an IEEE presentation with speaker notes.

Deliverable	Location	Purpose
Colab notebook	notebooks/traffic_sign_recognition_colab.ipynb	Standalone GPU execution
Project report	docs/traffic_sign_recognition_project_report.pdf	Detailed technical record
Synopsis	docs/traffic_sign_recognition_synopsis.pdf	Short project summary
Presenter guide	docs/traffic_sign_recognition_presenter_guide.pdf	Beginner explanation and viva preparation
Presentation	docs/traffic_sign_recognition_ieee_presentation.pptx	Presentation with speaker notes
Literature survey	docs/literature_survey.md	Verified sources and design mapping

12. Conclusion

The proposed project will study the complete machine-learning workflow for traffic-sign recognition, including data acquisition, exploratory analysis, leakage-aware validation, preprocessing, model training, GPU benchmarking, multi-metric evaluation, visual explanation, and technical communication. The study will compare a transparent custom CNN with a lightweight transfer-learning model and will document the resulting findings, limitations, and opportunities for further improvement.

13. References

[1] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, The German Traffic Sign Recognition Benchmark: A multi-class classification competition, IJCNN, 2011. https://www.ini.rub.de/upload/file/1470692848_f03494010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR, 2018. https://arxiv.org/abs/1801.04381

[3] D. P. Kingma and J. Ba, Adam: A Method for Stochastic Optimization, ICLR, 2015. https://arxiv.org/abs/1412.6980

[4] R. R. Selvaraju et al., Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization, ICCV, 2017. https://openaccess.thecvf.com/content_ICCV_2017/papers/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.pdf

[5] scikit-learn, GroupShuffleSplit documentation. https://scikit-learn.org/1.5/modules/generated/sklearn.model_selection.GroupShuffleSplit.html

[6] TensorFlow and Keras official documentation for tf.data, preprocessing layers, and callbacks. https://www.tensorflow.org/guide/data_performance and https://keras.io/api/callbacks/

[7] Maharaja Agrasen Institute of Technology, syllabus page. https://dev.mait.ac.in/new/index.php/portfolio/syllabus.html

[8] Maharaja Agrasen Institute of Technology, 7th Semester Minor Project Synopsis Submission Notice. https://cse.mait.ac.in/pdf/synopsis%20notice%20minor%20project%202023.pdf

[9] Maharaja Agrasen Institute of Technology, Department of Information Technology, student counselor record for Mr. Ajay Kaushik. https://it.mait.ac.in/index.php/it/counselor?id=541